

(520|600).666

Information Extraction from Speech and Text

Project # 3

Due May 7, 2026.

This project consists of two parts. In the first part, you will build an isolated-word speech recognizer just like you did for Project 2. In the second part, you will train a real speech recognition system for full sentences.

1 Isolated-word speech recognizer

In Project 2, you had built an isolated-word recognizer by first training a frame-level label prediction network, then running a maximum-likelihood decoding on all the word HMMs. Some of you might be wondering where the ground truth frame labels, which you relied on for training the label prediction network, originated from. If you had an inkling that a method which can give you accurate frame-level label predictions can probably also be used to perform speech recognition directly, then you would be right! In fact, we aligned the transcripts against the audio using a pre-trained speech recognition system.

As you can probably tell, this isn't an ideal situation. Relying on forced alignments generated from another system has many drawbacks, inefficiencies and potential for cascaded errors being some of them ¹. In order to get around the need for frame labels, the model needs to be trained so that it directly maximizes the likelihood of the direct transcript given the input features. This is where CTC comes in.

Your engineering tasks are as follows:

1. Implement CTC loss without using any external packages.
2. Modify the training objective so that your word recognizer is trained using CTC loss rather than frame-level cross-entropy loss. You don't need to break too much sweat over the neural network: your model from Project 2 is very likely good enough.
3. Produce a prediction for each of the test samples by picking the word that results in the lowest CTC loss given that sample, similar to the HMM-based maximum likelihood decoding in Project 2.

¹There are ways to design and train your network so that even if your alignments are inaccurate, the model can still train well and outperform the system used for generating your alignments. They're not the point of this exercise though.

4. Implement greedy decoding, and produce predictions using your trained model. Then measure the character error rate of your system.
5. (Optional) Implement beam search, and produce predictions using your trained model. Then measure the character error rate of your system.

Just like for Project 2, a PyTorch code skeleton will be provided.

Training and Test Data Files:

Please refer to the Project 2 handout for details.

2 Continuous Speech Recognition (CSR)

In this part of the project, we will build a complete automatic speech recognition (ASR) system for continuous speech using a pretrained self-supervised speech model, HuBERT².

For training, we will use PyTorch’s built-in **CTC loss** function, which is commonly used for sequence-to-sequence alignment in ASR. For data preparation and batching, we will use Lhotse³, a toolkit for working with speech datasets and building efficient training pipelines. **Note:** To complete this part, you will need access to a GPU. Please make a copy of the provided Colab notebook in your own Google Drive so that you can save your changes. Before switching to a GPU runtime, first make sure your code runs correctly on CPU to avoid wasting GPU time. Google Colab provides limited GPU access, typically for up to 12 hours at a time. If your session ends, you may need to wait 12–24 hours before GPU access becomes available again.

Suggestions:

- To use GPU time efficiently, first implement and test your code on CPU. Once your code runs without errors, switch to a GPU runtime for training.
- If you need longer or more reliable GPU access, you may consider upgrading to Colab Pro or Colab Pro+.
- Another possible workaround is to use multiple Google accounts and switch between them when GPU access is unavailable.

Proceed to build your *ASR* system using the code skeleton provided with Google colab here. Implement all sections marked **To do**.

1. **Character tokenizer:** Build the simplest text tokenizer (character-based tokenizer), that converts words into a sequence of IDs and can also map them back to text. Add the special token for padding `<pad>`, and pad each sequence to the maximum length.

²<https://arxiv.org/abs/2106.07447>

³<https://lhotse.readthedocs.io/>

Make sure to use `pad_id` to be different from `CTC blank_id`, which is 0 by default. In your report, explain your implementation and provide an example illustrating how it works.

2. **Training:** Modify the provided code and complete the HuBERT forward pass and the CTC forward pass. You should compute the CTC loss on the validation set during training to ensure that the model does not overfit. The training setup is already provided, but you still need to complete the training loop. Plot the training and validation losses, and discuss your observations. Report your final results on the `test-clean` set.
3. **CTC decoding:** Implement a simple greedy CTC decoder to generate predictions from the finetuned ASR model. Describe the logic of your implementation in your report.
4. **Analyze the results:** Use `jiwer`⁴ to analyze the results. Read the `jiwer` documentation to understand how to use it to identify the most frequent error types, i.e., insertions, deletions, and substitutions. Explain your observations; for example, what are the most common mistakes the model makes, and why do you think they occur? Are the substitution errors acceptable or acoustically reasonable mistakes.
5. **ASR with BPE:** Train a SentencePiece BPE tokenizer⁵ on the training transcripts to obtain a vocabulary of size 100, and save the vocabulary as a text file. Then, following Steps 1 to 4, implement a BPE tokenizer and train the speech recognition model using BPE units. Analyze the results and compare the errors with those of the character-based model. Which one performs better? In what ways is one better than the other in terms of substitutions, insertions, and deletions? Explain why one approach may provide these improvements. Report the results on the `test-clean` set and compare them with the ASR using a character tokenizer. Which system generalizes better? What is the effect of increasing BPE vocabulary size?

Submit a report answering the questions above and describing the rationale behind your ASR system implementation, together with the working code.

⁴<https://jitsi.github.io/jiwer/usage/>

⁵<https://github.com/google/sentencepiece/blob/master/python/README.md>